UNIVERSIDADE FEDERAL DE SERGIPE

Caminhos mínimos

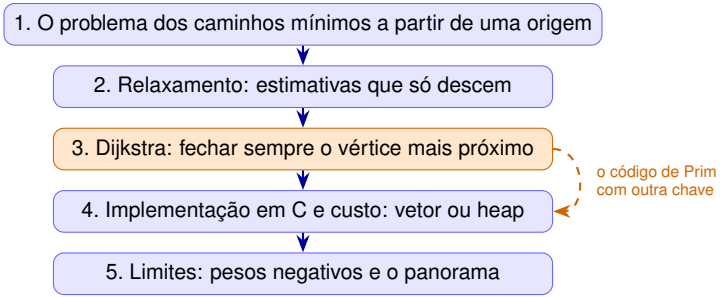
O algoritmo de Dijkstra

Prof. Dr. André Yoshiaki Kashiwabara

Departamento de Computação — Universidade Federal de Sergipe

10 de novembro de 2026

O que você vai aprender hoje



Na aula passada, Prim trazia para a árvore o vértice de *aresta* mais barata. Hoje, Dijkstra traz o vértice de *caminho* mais barato.

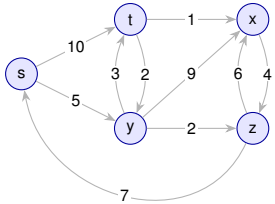
Para aprofundar: Cormen et al. (2009), cap. 24 [1]; Kleinberg e Tardos (2006), §4.4 [8].

O problema dos caminhos mínimos

Por que precisamos de caminhos mínimos?

Problema: um aplicativo de mapas precisa da rota mais rápida de Aracaju até cada cidade de Sergipe. As vias são de mão única ou dupla, e cada trecho tem um tempo de percurso.

A busca em largura (BFS) acha o caminho com *menos trechos*, mas um trecho pode levar 2 minutos e outro, 40.



Exemplo condutor: origem s. Menos arcos: $s \to t \to x$ (custo 11). Menor custo: $s \to y \to t \to x$ (custo 9).

Para aprofundar: Cormen et al. (2009), §24.3, Figura 24.6 [1]; Dasgupta et al. (2008), §4.4 [3].

Caminhos mínimos: definição

Definição

Seja $G = (V, E)$ um digrafo com custos $w(u, v)$ nos arcos. O **custo** de um caminho $P = \langle v_0, v_1, \ldots, v_k \rangle$ é $w(P) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$. A **distância** de s a v é

$$\delta(s, v) = \min\{\, w(P) : P \text{ é caminho de } s \text{ a } v \,\},$$

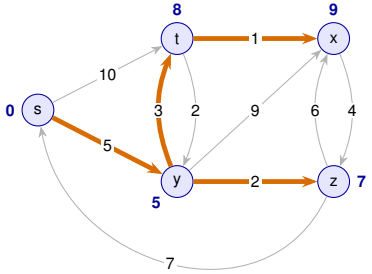
com $\delta(s, v) = \infty$ se v não é alcançável a partir de s.

Intuição

Problema de **origem única**: uma origem s fixa, a distância a *todos* os vértices de uma vez. Achar só $s \to v$ não é mais fácil no pior caso.

Notação: $n = |V|$, $m = |E|$. Grafo não dirigido: cada aresta vira dois arcos. Hoje, todos os custos são ≥ 0.
Para aprofundar: Cormen et al. (2009), introdução do cap. 24 [1]; Kleinberg e Tardos (2006), §4.4 [8].

Exemplo: as distâncias no digrafo condutor



v	caminho mínimo	$\delta(s, v)$
s	s	0
y	$s \to y$	5
z	$s \to y \to z$	7
t	$s \to y \to t$	8
x	$s \to y \to t \to x$	9

Em azul, $\delta(s, v)$. Os arcos em laranja formam uma **árvore de caminhos mínimos**: o caminho de s a cada v dentro dela é mínimo. Note que o arco $s \to t$, de custo 10, não serve para nada.

Para aprofundar: Cormen et al. (2009), §24.3, Figura 24.6(f) [1].

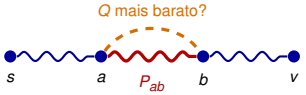
A subestrutura ótima dos caminhos mínimos

Lema

Todo trecho de um caminho mínimo é um caminho mínimo.

Prova (recorte e cola). Seja P mínimo de s a v e P_{ab} o trecho de a a b. Se houvesse Q de a a b com $w(Q) < w(P_{ab})$, trocar P_{ab} por Q em P daria um caminho de s a v mais barato que P: absurdo. □

Consequência. Basta guardar, para cada v, o **predecessor** $pai[v]$ no caminho mínimo: os $n-1$ arcos $(pai[v], v)$ formam a árvore da página anterior.



É a **subestrutura ótima** das aulas de programação dinâmica; Floyd já dependia dela.

Para aprofundar: Cormen et al. (2009), cap. 24, Lema 24.1 [1]; Kleinberg e Tardos (2006), §4.4 [8].

O que já sabemos fazer

algoritmo	resolve	restrição	tempo
força bruta	enumera todos os caminhos	—	exponencial
BFS (Unidade 1)	origem única	todo custo $= 1$	$O(n + m)$
Floyd (Unidade 2)	*todos os pares*	sem ciclo negativo	$\Theta(n^3)$
Dijkstra (hoje)	origem única	custos ≥ 0	$O(m \log n)$

Floyd responde à pergunta de origem única, mas paga $\Theta(n^3)$ calculando n^2 distâncias quando queremos só n.

Num mapa de ruas com $n = 10^6$ cruzamentos, $n^3 = 10^{18}$ operações; o grafo é esparso ($m \approx 3n$), e $m \log n \approx 6 \times 10^7$.

Para aprofundar: Cormen et al. (2009), cap. 24 e §25.2 [1]; Dasgupta et al. (2008), §4.4 [3].

Recapitulando: o problema

O que aprendemos

- $\delta(s, v)$ é o menor custo de um caminho de s a v (∞ se não há).
- Origem única: queremos $\delta(s, v)$ para todo v de uma vez.
- Trechos de caminhos mínimos são mínimos: basta guardar $pai[v]$.
- Os predecessores formam uma árvore de caminhos mínimos.
- BFS só vale para custos unitários; Floyd é caro demais para uma origem.

Para aprofundar: Cormen et al. (2009), cap. 24 [1].

Relaxamento

Por que trabalhar com estimativas?

Problema: não conhecemos $\delta(s, v)$ de antemão. Precisamos de uma operação que melhore aos poucos o que sabemos, sem nunca errar para baixo.

Analogia: o preço de uma passagem que você vai pesquisando. Cada rota nova encontrada só pode *baixar* o melhor preço conhecido.

Para aprofundar: Cormen et al. (2009), cap. 24, “Relaxation” [1].

Solução: para cada v, uma **estimativa** $dist[v]$ que é o custo de *algum* caminho de s a v já encontrado:

$$\boxed{dist[v] \geq \delta(s, v) \text{ sempre}}$$

Começa com $dist[s] = 0$ e ∞ no resto, e só desce.

Dijkstra, Bellman-Ford e o algoritmo para DAGs usam a mesma operação; o que muda é a *ordem* em que a aplicam.

Relaxar um arco: definição

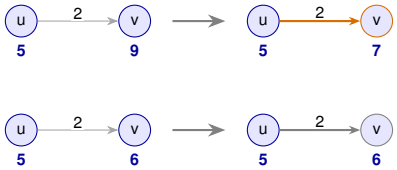
RELAXA(u, v)

se $dist[u] + w(u, v) < dist[v]$:
$dist[v] \leftarrow dist[u] + w(u, v)$
$pai[v] \leftarrow u$

Intuição

"Chegar a v passando por u é melhor do que o melhor caminho que eu conhecia?" Se sim, adote o novo caminho: o de u seguido do arco $u \rightarrow v$.

Para aprofundar: Cormen et al. (2009), cap. 24, RELAX e Figura 24.3 [1].



Em cima, $5 + 2 < 9$: $dist[v]$ cai para 7 e $pai[v] = u$.
Embaixo, $5 + 2 \geq 6$: nada muda.

Propriedades do relaxamento

Desigualdade triangular. Para todo arco (u, v): $\delta(s, v) \leq \delta(s, u) + w(u, v)$.

Limite superior. Em qualquer sequência de relaxamentos, $dist[v] \geq \delta(s, v)$, e quando $dist[v]$ atinge $\delta(s, v)$ não muda mais.

Convergência. Se $s \rightsquigarrow u \to v$ é um caminho mínimo e $dist[u] = \delta(s, u)$ no momento em que (u, v) é relaxado, então depois disso $dist[v] = \delta(s, v)$.

Relaxamento de caminho. Se os arcos de um caminho mínimo $\langle s, v_1, \ldots, v_k \rangle$ forem relaxados *nessa ordem* (com quaisquer outros relaxamentos no meio), no fim $dist[v_k] = \delta(s, v_k)$.

A pergunta da aula

Em que ordem relaxar os arcos para que cada arco seja relaxado **uma vez só**?

Para custos ≥ 0, a resposta é de Dijkstra.

Para aprofundar: Cormen et al. (2009), §24.5, Lemas 24.10, 24.11, 24.14 e 24.15 [1].

Recapitulando: relaxamento

O que aprendemos

- $dist[v]$ é o custo de um caminho conhecido: nunca menor que $\delta(s, v)$.
- Relaxar (u, v) testa se passar por u melhora o caminho até v.
- Relaxar os arcos de um caminho mínimo em ordem acerta a distância do fim.
- Os algoritmos de caminho mínimo diferem na ordem dos relaxamentos.

Para aprofundar: Cormen et al. (2009), cap. 24 e §24.5 [1].

O algoritmo de Dijkstra

Por que fechar o vértice mais próximo?

Problema: relaxar um arco (u, v) só é definitivo se $dist[u]$ já for exata. Qual vértice tem certamente a estimativa exata?

Imagine a origem soltando água que escorre pelos arcos com velocidade constante: os vértices molham em ordem crescente de distância. O mais próximo dos ainda secos é o próximo a molhar.

Solução (Dijkstra, 1959): manter o conjunto S de vértices **fechados**, cuja distância já é exata.

1. $S \leftarrow \emptyset$; $dist[s] = 0$;
2. escolha $u \notin S$ com **menor** $dist[u]$ e feche u;
3. relaxe os arcos que saem de u;
4. repita até fechar todos os alcançáveis.

Cada arco é relaxado uma vez só: quando sua ponta inicial fecha.

Para aprofundar: Dijkstra (1959) [4]; Dasgupta et al. (2008), §4.4 (adaptação da BFS) [3]; Levitin (2012), §9.3 [9].

O algoritmo de Dijkstra

DIJKSTRA(G, w, s)

para cada v: $dist[v] \leftarrow \infty$; $pai[v] \leftarrow -1$
$dist[s] \leftarrow 0$; $S \leftarrow \emptyset$
$Q \leftarrow$ fila de prioridade com V, chave $dist$
enquanto $Q \neq \emptyset$:
  $u \leftarrow$ EXTRAI-MIN(Q); $S \leftarrow S \cup \{u\}$
  para cada arco (u, v):
    RELAXA(u, v)

Compare com Prim, da aula passada: a estrutura é a mesma, só a chave muda.

	a chave de v baixa para
Prim	$w(u, v)$
Dijkstra	$dist[u] + w(u, v)$

Prim pergunta "quanto custa *pendurar* v na árvore?"; Dijkstra pergunta "quanto custa *chegar* a v desde s?". Com custos unitários e uma fila comum no lugar da de prioridade, Dijkstra vira a BFS.

Para aprofundar: Cormen et al. (2009), §24.3, DIJKSTRA [1]; Kleinberg e Tardos (2006), §4.4 [8].

Exemplo: Dijkstra a partir de s

fecha	s	t	x	y	z
—	0	∞	∞	∞	∞
s (0)	–	**10**	∞	**5**	∞
y (5)	–	**8**	**14**	–	**7**
z (7)	–	8	**13**	–	–
t (8)	–	–	**9**	–	–
x (9)	–	–	–	–	–

dist depois de cada fechamento; em negrito, as que baixaram.

Fecha y: t cai de 10 para $5 + 3 = 8$, pois chegar a t por y é mais barato que pelo arco direto.

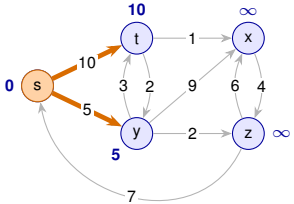
Fecha z: x cai de 14 para $7 + 6 = 13$. O arco $z \to s$ não muda nada: s já está fechado.

Fecha t: x cai de novo, para $8 + 1 = 9$. Ao fechar x, todos os relaxamentos falham.

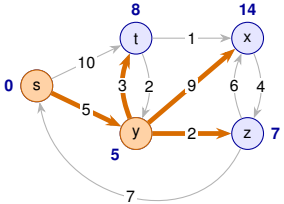
Ordem de fechamento: s, y, z, t, x, com distâncias **crescentes** $0, 5, 7, 8, 9$.

Para aprofundar: Cormen et al. (2009), §24.3, Figura 24.6 [1].

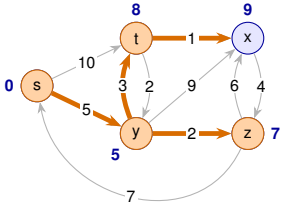
As estimativas ao longo do tempo



depois de fechar s



depois de fechar y



depois de fechar t

Em laranja, os vértices fechados e os arcos $(pai[v], v)$ do momento. O arco $s \to t$ entra na árvore e depois sai, quando y oferece um caminho melhor até t; o mesmo acontece com $y \to x$.

Para aprofundar: Cormen et al. (2009), §24.3, Figura 24.6(b)–(e) [1].

Dijkstra: por que está correto

Teorema

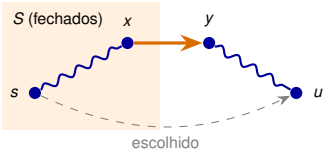
Com custos $w \geq 0$, ao ser fechado, cada vértice u tem $dist[u] = \delta(s, u)$.

Prova. Suponha que u seja o primeiro a fechar com $dist[u] > \delta(s, u)$. Tome um caminho mínimo P de s a u e seja y o primeiro vértice de P fora de S, com predecessor $x \in S$.

x fechou com $dist[x] = \delta(s, x)$ e relaxou (x, y); por convergência, $dist[y] = \delta(s, y)$. Como y vem antes de u em P e os custos são ≥ 0:

$$dist[y] = \delta(s, y) \leq \delta(s, u) < dist[u].$$

Mas o algoritmo escolheu u, e não y: $dist[u] \leq dist[y]$. Absurdo. □



Em azul e laranja, o caminho mínimo P. É aqui que $w \geq 0$ é indispensável: o trecho de y a u não pode diminuir o custo.

Para aprofundar: Cormen et al. (2009), §24.3, Teorema 24.6 e Figura 24.7 [1]; Kleinberg e Tardos (2006), §4.4, (4.14) [8].

Recapitulando: Dijkstra

O que aprendemos

- Dijkstra fecha sempre o vértice aberto de menor *dist*.
- Ao fechar u, relaxa os arcos que saem de u; cada arco, uma vez.
- É o código de Prim com chave $dist[u] + w(u, v)$ no lugar de $w(u, v)$.
- Os vértices fecham em ordem crescente de distância.
- A correção depende de $w \geq 0$: nenhum caminho fica mais barato ao crescer.

Para aprofundar: Cormen et al. (2009), §24.3 [1].

Implementação e custo

Dijkstra em C com matriz de adjacência

```c
/* W[u][v] = custo do arco u->v, ou INF se nao ha arco */
void dijkstra( int n, int W[][MAXN], int s, int dist[], int pai[]) {
   bool fechado[MAXN];
   for (int v = 0; v < n; ++v) {
      dist[v] = INF;  pai[v] = -1;  fechado[v] = false;
   }
   dist[s] = 0;
   for (int it = 0; it < n; ++it) {
      int u = -1;                          /* o aberto mais proximo */
      for (int v = 0; v < n; ++v)
         if (!fechado[v] && (u < 0 || dist[v] < dist[u]))
            u = v;
      if (dist[u] == INF) break;           /* o resto e inalcancavel */
      fechado[u] = true;
      for (int v = 0; v < n; ++v)          /* relaxa os arcos de u */
         if (W[u][v] != INF && dist[u] + W[u][v] < dist[v]) {
            dist[v] = dist[u] + W[u][v];  pai[v] = u;
         }
   }
}
```

Para aprofundar: Feofiloff (2020), algoritmo de Dijkstra [5]; Levitin (2012), §9.3 [9].

Diferenças para o Prim da aula passada:

- a chave é $dist[u] + W[u][v]$;
- o teste `W[u][v] != INF` evita somar com `INF` e transbordar o `int`;
- se o mais próximo tem $dist = \infty$, os demais são inalcançáveis.

Custo. n iterações, cada uma com duas varreduras de n posições: $\Theta(n^2)$, ótimo para grafos densos.

Reconstruindo o caminho com `pai[]`

```
/* imprime o caminho minimo de s a t, de s para t */
void imprime_caminho( int pai[], int s, int t) {
   if (t == s) { printf( "%d", s);  return; }
   if (pai[t] == -1) {
      printf( "%d inalcancavel", t);  return;
   }
   imprime_caminho( pai, s, pai[t]);
   printf( " -> %d", t);
}
```

A recursão sobe de t até s e imprime na volta, na ordem certa. Custo: o número de arcos do caminho.

v	s	t	x	y	z
$pai[v]$	-1	y	t	s	y

Caminho até x: $pai[x] = t$, $pai[t] = y$, $pai[y] = s$. Lido de trás para frente: $s \to y \to t \to x$, custo $5 + 3 + 1 = 9$.

O vetor *pai* guarda a árvore de caminhos mínimos inteira em n posições: os n caminhos juntos podem ter $\Theta(n^2)$ arcos.

Para aprofundar: Cormen et al. (2009), §22.2, PRINT-PATH, e cap. 24, “Representing shortest paths” [1].

Quanto custa Dijkstra?

fila de prioridade	operações dominantes	total
vetor (varredura)	n extrações de $\Theta(n)$	$\Theta(n^2)$
heap binário	n extrações e até m reduções de chave, $O(\log n)$ cada	$O((n+m)\log n)$
heap de Fibonacci	redução de chave em $O(1)$ amortizado	$O(m + n\log n)$

Grafo esparso ($m = O(n)$): heap binário dá $O(n \log n)$; o vetor perde com $\Theta(n^2)$.

Grafo denso ($m = \Theta(n^2)$): o vetor dá $\Theta(n^2)$; o heap binário perde com $O(n^2 \log n)$.

Heap preguiçoso. Sem redução de chave: a cada relaxamento bem-sucedido, insira o par $(dist[v], v)$ de novo; ao extrair um par com custo maior que $dist[v]$, descarte-o. O heap chega a m pares, e o custo fica $O(m \log n)$. É o Exercício 2 do tutorial.

Para aprofundar: Cormen et al. (2009), §24.3 (análise) [1]; Fredman e Tarjan (1987) [6]; Dasgupta et al. (2008), §4.5 [3].

Recapitulando: implementação

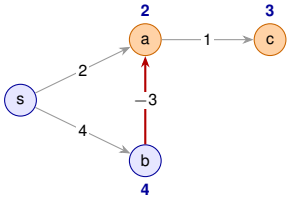
O que aprendemos

- Com matriz e varredura, Dijkstra custa $\Theta(n^2)$: bom para grafos densos.
- Com listas de adjacência e heap binário, $O((n+m)\log n)$: bom para esparsos.
- O heap preguiçoso dispensa a redução de chave e descarta pares velhos.
- O vetor *pai* guarda todos os caminhos em n posições.

Para aprofundar: Cormen et al. (2009), §24.3 [1].

Limites e panorama

Por que custos negativos quebram Dijkstra?



Dijkstra fecha s (0), a (2), c (3) e por último b (4). Ao fechar b, o arco $b \to a$ ofereceria $4 - 3 = 1$ para a, mas a e c já estão fechados.

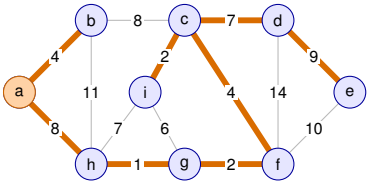
Resposta errada: $dist[c] = 3$, mas $s \to b \to a \to c$ custa $4 - 3 + 1 = 2$.

A prova falha exatamente no passo $\delta(s, y) \leq \delta(s, u)$: um trecho com custo negativo pode deixar o caminho mais barato ao crescer.

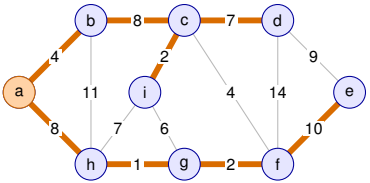
E somar uma constante a todos os custos? Não funciona: penaliza mais os caminhos com mais arcos. Somando 3, $s \to a \to c$ custa 9 e $s \to b \to a \to c$ custa 11, e o mínimo muda.

Para aprofundar: Cormen et al. (2009), cap. 24, “Negative-weight edges”, e Exercício 24.3-2 [1]; Dasgupta et al. (2008), §4.6 [3].

Árvore de caminhos mínimos não é AGM



AGM de Kruskal: peso **37**. De *a* a *e*: *a h g f c d e*, custo 31.



Dijkstra a partir de *a*: peso **42**. De *a* a *e*: *a h g f e*, custo 21.

O grafo da aula passada. A AGM minimiza a **soma de todas as arestas**; a árvore de Dijkstra minimiza **cada caminho a partir de** *a*. Os objetivos são diferentes e as árvores também: a de Dijkstra troca (c, f) e (d, e) por (b, c) e (e, f).

Para aprofundar: Cormen et al. (2009), §23.1, Figura 23.1, e §24.3 [1].

O panorama dos caminhos mínimos

algoritmo	quando usar	ordem dos relaxamentos	tempo
BFS	custos unitários	fila comum	$O(n+m)$
DAG	digrafo acíclico, qualquer custo	ordem topológica	$O(n+m)$
Dijkstra	custos ≥ 0	menor *dist* primeiro	$O(m \log n)$
Bellman-Ford	custos negativos; detecta ciclo negativo	todos os arcos, $n-1$ vezes	$O(nm)$
Floyd	todos os pares	intermediários $0..k-1$	$\Theta(n^3)$
Johnson	todos os pares, esparso	Bellman-Ford + $n\times$ Dijkstra	$O(nm \log n)$

Todos, exceto Floyd, são o mesmo relaxamento em ordens diferentes. A ordem topológica da Unidade 1 também resolve caminhos mínimos, e até *máximos*, em DAGs. Na prática, GPS e roteadores usam Dijkstra e variantes com heurística, como o A* (desafio do tutorial).

Para aprofundar: Cormen et al. (2009), §24.1, §24.2, §24.3 e §25.3 [1]; Hart, Nilsson e Raphael (1968), A* [7].

Fechando

O que levar desta aula

- Relaxar arcos só baixa estimativas, que nunca ficam abaixo de δ.
- Dijkstra relaxa a partir do vértice aberto mais próximo, uma vez por arco.
- É guloso: fecha cada vértice sem voltar atrás, como Prim.
- Exige custos ≥ 0; para negativos, Bellman-Ford.
- $\Theta(n^2)$ com vetor, $O(m \log n)$ com heap.

Para aprofundar: Misa e Frana (2010) [10]; Dijkstra (1959) [4].

História. Dijkstra conta que concebeu o algoritmo em 1956, em uns vinte minutos, sentado num café em Amsterdã, sem papel e lápis, para demonstrar um computador novo calculando rotas entre 64 cidades holandesas. Publicou três anos depois, numa nota de três páginas que também resolvia a AGM.

Próxima aula: fluxo máximo e emparelhamento, em que caminhos num grafo *residual* são usados para aumentar o fluxo.

Referências I



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, 3 edition, 2009.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Algoritmos: Teoria e Prática.
Elsevier, Rio de Janeiro, 3 edition, 2012.
Tradução da 3. ed. norte-americana. Seção 4.5.



Sanjoy Dasgupta, Christos H. Papadimitriou, and Umesh V. Vazirani.
Algorithms.
McGraw-Hill, 2008.
Seção 2.1: Multiplication (o truque de Gauss).



Edsger W. Dijkstra.
A note on two problems in connexion with graphs.
Numerische Mathematik, 1:269–271, 1959.



Paulo Feofiloff.
Algoritmos para grafos (em linguagem C).
Material didático online, DCC/IME-USP, 2020.
Capítulos sobre árvores geradoras de custo mínimo e os algoritmos de Prim e de Kruskal. Acesso em 30 out. 2026.

Referências II



Michael L. Fredman and Robert Endre Tarjan.

Fibonacci heaps and their uses in improved network optimization algorithms.
Journal of the ACM, 34(3):596–615, 1987.



Peter E. Hart, Nils J. Nilsson, and Bertram Raphael.

A formal basis for the heuristic determination of minimum cost paths.
IEEE Transactions on Systems Science and Cybernetics, 4(2):100–107, 1968.



Jon Kleinberg and Éva Tardos.

Algorithm Design.
Pearson/Addison-Wesley, 2006.
Capítulo 5, em especial as seções 5.2 (recorrências) e 5.4 (par mais próximo).



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.
Pearson, 3 edition, 2012.



Thomas J. Misa and Philip L. Frana.

An interview with Edsger W. Dijkstra.
Communications of the ACM, 53(8):41–47, 2010.

Referências III



Robert Sedgewick and Kevin Wayne.
Algorithms.
Addison-Wesley, 4 edition, 2011.